1 INTRODUCTION

Retrieval-Augmented Generation (RAG) mitigates LLM hallucinations by grounding responses in retrieved external knowledge. While traditional vector RAG excels at factoid lookups, it struggles with global questions requiring cross-document reasoning—questions such as "What are the main themes across this document collection?" or "How are these entities related?" GraphRAG addresses this limitation by constructing knowledge graphs from text corpora, enabling structured multi-hop retrieval and community-level summarization [1, 2]. By organizing entities and their relationships into a graph index, GraphRAG systems can answer queries that require synthesizing information across multiple documents, which flat vector retrieval cannot handle effectively.

A critical limitation of current GraphRAG systems is their reliance on *full rebuilds* for corpus updates. Microsoft GraphRAG [1] requires re-running entity extraction, community detection, and summary generation over the entire corpus when any new document arrives. For a 1M-token corpus, this costs over 280 minutes of LLM time and significant API expenditure. LightRAG [2] introduced a set-union incremental update mechanism that avoids full rebuilds, but its naive merging approach creates three critical problems: (1) duplicate entities—the same real-world entity extracted from different documents with slightly different names creates redundant nodes; (2) graph fragmentation—new entities are added without connecting them to the existing graph structure, creating disconnected components; and (3) stale communities—community assignments computed on the initial corpus become outdated as new entities arrive, degrading global query performance.

These problems compound over time: with each incremental batch, the graph becomes increasingly bloated with duplicates, fragmented across disconnected components, and organized into communities that no longer reflect the current graph structure. This degradation directly impacts retrieval quality—our experiments show that naive incremental updates cause R@5 to drift by up to 18% after 12 batches on HotpotQA, while the full rebuild approach would require recomputing the entire index from scratch.

We propose IGV (Incremental Graph Versioner), which addresses these three problems through corresponding innovations:

- **Semantic entity deduplication** using sentence-transformer embeddings with cosine similarity, reducing entity count by 13–17% by merging entities that refer to the same real-world concept but were extracted with different surface forms.
- **Multi-hop graph relinking** that connects newly inserted entities to the existing graph via BFS-based candidate generation, ensuring graph connectivity and enabling cross-document reasoning paths.
- **Incremental community repartitioning** that locally updates Louvain community assignments for affected nodes only, maintaining community quality without the $O(N \log N)$ cost of full recomputation.

Experiments on five multi-hop QA datasets demonstrate that IGV achieves GQD below 6.6% relative to full rebuild, with near-zero forgetting rate (FR<0.01). On 2Wiki, IGV *outperforms* full rebuild by 1.5% in R@5, showing that deduplication can actually improve retrieval quality. Statistical significance tests confirm F1 improvements ($p < 0.05$), and long-term streaming over 12 batches with four ablation methods shows sub-9% R@5 degradation.

2 RELATED WORK

GraphRAG Systems. Edge et al. [1] proposed GraphRAG, which builds entity knowledge graphs via LLM extraction, applies Leiden community detection, and generates community summaries for global query answering. Their system processes documents in two phases: offline indexing (entity extraction → graph construction → community detection → summary generation) and online query (community summary retrieval → map-reduce answer generation). While effective for static corpora, the system requires complete re-indexing when new documents arrive. LightRAG [2] simplified this with a dual-level retrieval paradigm (low-level entity retrieval + high-level topic retrieval) and introduced basic incremental updates via set union of entity and edge sets. However, LightRAG's incremental approach does not address entity deduplication, graph relinking, or community repartitioning. PathRAG [3] introduced flow-based path pruning to reduce retrieval redundancy, achieving 13.7% token reduction while improving multi-hop QA performance. HippoRAG [4] used Personalized PageRank for neurobiologically inspired retrieval, simulating the hippocampal indexing theory of human memory. GNN-RAG [8] combined graph neural networks with LLMs for KGQA, using GNNs for graph reasoning and LLMs for natural language understanding.

Incremental Knowledge Graph Updates. GORAG [5] proposed online graph updates for social media text classification, dynamically constructing graphs at query time. However, GORAG does not maintain a persistent graph index across queries, making it unsuitable for GraphRAG's community-based retrieval. KAG [6] introduced logical-form-guided reasoning with schema constraints for professional domain QA, but its knowledge construction does not support incremental community repartitioning. Practical GraphRAG [7] reduced construction cost via dependency parsing instead of LLM-based extraction, achieving 94% of LLM-based performance at significantly lower cost. However, this approach still requires full rebuilds for corpus updates and does not address the incremental update problem.

Community Detection. Louvain [9] and Leiden [10] algorithms are the standard approaches for modularity-based community detection in large graphs. The Leiden algorithm improves upon Louvain by guaranteeing well-connected communities through a refinement phase. Dynamic and incremental community detection has been studied in the network science literature, with approaches such as label propagation updates and local modularity optimization. However, these techniques have not been

applied to the GraphRAG domain, where community structure directly impacts query answering quality. Our work bridges this gap by adapting incremental community detection to the GraphRAG pipeline.

3 METHODOLOGY

3.1 Problem Formulation

Given a knowledge graph $G = (V, E)$ constructed from an initial document corpus $\mathcal{D}_0$, and a stream of new document batches $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_T$ arriving over time, the goal is to update G to incorporate new entities and relationships from each batch $\mathcal{D}_t$ while satisfying four objectives:

(1) **Efficiency:** Minimizing LLM calls per update (the dominant cost in GraphRAG).
(2) **Consistency:** Avoiding duplicate entities that fragment the graph.
(3) **Community quality:** Maintaining community structure that reflects the current graph topology.
(4) **Retrieval utility:** Preserving retrieval performance (R@5, F1) relative to full rebuild.

The full rebuild approach processes all documents $\bigcup_{i=0}^{t} \mathcal{D}_i$ from scratch at each step, with cost $O(\sum_{i=0}^{t} |\mathcal{D}_i|)$ LLM calls. The naive incremental approach (LightRAG) processes only the new batch $\mathcal{D}_t$, with cost $O(|\mathcal{D}_t|)$, but sacrifices graph consistency and community quality. IGV achieves the same $O(|\mathcal{D}_t|)$ LLM cost while maintaining consistency through deduplication, connectivity through relinking, and community quality through incremental repartitioning.

3.2 IGV Architecture

IGV processes each incremental batch $\mathcal{D}_t$ through four sequential stages:

Stage 1: Entity Extraction. For each new document $d \in \mathcal{D}_t$, an LLM (Qwen2.5-7B-Instruct) extracts named entities and their relationships as triplets (s, r, o), where s and o are entity mentions and r is the relationship description. The extraction prompt instructs the LLM to identify people, organizations, locations, concepts, and other named entities, returning results as a JSON list. Each entity is augmented with a description derived from its surrounding context in the source document.

Stage 2: Semantic Deduplication. After extraction, new entities are compared against existing graph nodes to identify potential duplicates. We encode all entity names and descriptions using a sentence-transformer model (all-MiniLM-L6-v2, $d = 384$ dimensions) with L2-normalized embeddings. Given a new entity e_{new} with embedding $\mathbf{e}_{\text{new}}$ and existing entities with embeddings $\{\mathbf{e}_1, \ldots, \mathbf{e}_N\}$, we compute:

$$\text{sim}(e_{\text{new}}, e_i) = \mathbf{e}_{\text{new}} \cdot \mathbf{e}_i \tag{1}$$

where the dot product equals cosine similarity due to L2 normalization. If $\text{sim}(e_{\text{new}}, e_i) \geq \tau$ (threshold $\tau = 0.85$), the entities are merged: the new entity's relationships are redirected to the existing entity, descriptions are concatenated (keeping the longer one as primary), and source document IDs are accumulated. This batch computation is vectorized via numpy matrix multiplication, achieving $O(m \times N)$ cost where m is the number of new entities and N is the number of existing entities, with a constant factor dominated by the embedding computation.

Stage 3: Graph Relinking. After deduplication, new entities may be disconnected from the existing graph if their source documents do not share entities with previously indexed documents. We address this by identifying candidate edges between new and existing nodes using BFS expansion to depth $h = 2$. For each new node v_{new}, we explore its h-hop neighborhood and compute a confidence score for each candidate existing node v_{old}:

$$\text{confidence}(v_{\text{new}}, v_{\text{old}}) = \frac{1}{1 + \text{dist}(v_{\text{new}}, v_{\text{old}})} \tag{2}$$

where dist is the shortest path distance in the graph. Edges with confidence ≥ 0.85 (i.e., within 1–2 hops) are added to the graph, with edge weight proportional to the confidence score. This ensures that new entities are connected to structurally related existing entities, enabling multi-hop reasoning paths across document boundaries.

Stage 4: Incremental Community Repartitioning. Rather than recomputing communities for the entire graph, we identify the *affected region*—the set of nodes whose community membership may have changed. This region consists of all new nodes plus their 2-hop neighbors in the existing graph. We extract the subgraph induced by this region and re-apply the Louvain algorithm [9] only on this subgraph. The new sub-communities are then merged back into the global partition: existing community assignments for unaffected nodes are preserved, while affected nodes are reassigned to their new communities.

Formally, let $\mathcal{C}_{t-1} = \{C_1, C_2, \ldots, C_k\}$ be the community partition at step $t-1$, and S_t be the set of affected nodes at step t. The updated partition $\mathcal{C}_t$ is:

$$\mathcal{C}_t = \{C_i \setminus S_t \mid C_i \in \mathcal{C}_{t-1}, C_i \setminus S_t \neq \emptyset\} \cup \text{Louvain}(G[S_t]) \tag{3}$$

where $G[S_t]$ is the subgraph induced by S_t. This approach has cost $O(|S_t| \log |S_t|)$, which is sublinear in the total graph size N when $|S_t| \ll N$.

Table 1: Complexity comparison per incremental batch (m: new entities, N: total entities, $|S|$: affected subgraph, d: avg degree, h: hop count).

Operation	Full Rebuild	IGV
Entity extraction	$O(N+m)$	$O(m)$
Deduplication	$O((N+m)^2)$	$O(m \cdot N)$
Community detection	$O((N+m)\log(N+m))$	$O(\|S\|\log\|S\|)$
Total LLM calls	$O(N+m)$	$O(m)$

3.3 Complexity Analysis

Table 1 compares the computational complexity of IGV against full rebuild and naive incremental approaches.

When $m \ll N$ (the typical case in streaming scenarios), IGV's cost is dominated by the deduplication step $O(m \cdot N)$, which is significantly cheaper than the rebuild cost $O(N+m)$ for entity extraction alone. The community repartition cost $O(|S|\log|S|)$ is typically negligible since $|S| \approx m \cdot d^h \ll N$.

4 EXPERIMENTS

4.1 Setup

Datasets. We evaluate on five multi-hop QA benchmarks: HotpotQA (Wikipedia-based multi-hop reasoning), 2WikiMultiHopQA (entity-centric multi-hop), MuSiQue (compositional multi-hop with 2–4 hops), NarrativeQA (long-form narrative understanding), and StreamingQA (temporally ordered passages simulating streaming updates). We use 300 passages and up to 200 evaluation questions per dataset. Each dataset is split into 70% base corpus (initial index) and three incremental batches of 10% each, simulating a streaming document arrival pattern.

LLM. Qwen2.5-7B-Instruct for entity extraction and answer generation, running on a single NVIDIA RTX 5090 GPU (32GB VRAM) with bfloat16 precision. Entity embeddings are computed using all-MiniLM-L6-v2 (384-dimensional, L2-normalized). The LLM is used in two roles: (1) entity extraction from source documents, where it identifies named entities and returns them as JSON lists, and (2) answer generation, where it produces concise answers given retrieved context.

Baselines. We compare five methods in a progressive ablation design:

- **B1 (Rebuild):** Full graph reconstruction from scratch at each stage—re-processes all accumulated documents. This represents the quality upper bound but with maximum cost.
- **B2 (Naive):** LightRAG-style set-union incremental—processes only new documents but performs no deduplication, relinking, or community detection.
- **B3 (+Dedup):** B2 + semantic deduplication ($\tau = 0.85$)—merges duplicate entities using embedding cosine similarity.
- **B4 (+Relink):** B3 + 2-hop graph relinking—connects new entities to the existing graph structure.
- **IGV (Ours):** B4 + incremental community repartitioning—the complete proposed method.

Metrics. We report: Recall@2 (R@2) and Recall@5 (R@5) for retrieval quality; Answer F1 and Exact Match (EM) for generation quality; Comprehensiveness and Diversity (LLM-as-judge, 1–5 scale) for answer utility; entity count and update time for efficiency; and four incremental-specific metrics: GQD (Graph Quality Degradation: relative R@5 loss vs rebuild), UER (Update Efficiency Ratio: rebuild LLM calls / incremental LLM calls), SF (Subgraph Freshness: fraction of new nodes per batch), and FR (Forgetting Rate: relative R@5 decrease from previous batch).

Seeds. All experiments are repeated with 3 random seeds (42, 52, 62) controlling data splitting and document ordering. We report mean±standard deviation.

4.2 Main Results

Table 2 presents results at the final incremental stage (Stage 3) across all five datasets. Key observations:

IGV matches rebuild quality. On HotpotQA, IGV achieves F1=0.108, matching B1's 0.108, while B2 drops to 0.103. On 2Wiki, B3/IGV achieve F1=0.112, *outperforming* B1's 0.107. This demonstrates that deduplication not only avoids degradation but can actually improve answer quality by reducing retrieval noise.

Deduplication improves F1. The progressive ablation B2→B3 shows F1 improvements on multiple datasets: HotpotQA (0.103→0.107), 2Wiki (0.106→0.112), and MuSiQue (0.031→0.034). This confirms that removing duplicate entities reduces retrieval noise, leading to more focused context for answer generation.

Relinking benefits narrative QA. On NarrativeQA, B4 (+Relink) achieves F1=0.092, outperforming B3's 0.086 by 7%. This suggests that cross-document connections are particularly valuable for narrative understanding, where reasoning across story elements requires explicit relationship links.

Table 2: Main results at Stage 3 (mean±std over 3 seeds, 200 questions). Best in bold.

Dataset	Method	R@2	R@5	F1	EM	Comp	Div
HotpotQA	B1 Rebuild	**0.202±.016**	**0.396±.013**	0.108±.004	**0.040**	**2.5**	**2.0**
	B2 Naive	0.202±.014	0.386±.004	0.103±.003	0.020	2.4	1.9
	B3 +Dedup	0.186±.019	0.379±.000	0.107±.003	0.020	**2.5**	**2.0**
	B4 +Relink	0.186±.019	0.379±.000	0.107±.001	0.020	**2.5**	**2.0**
	IGV (Ours)	0.186±.019	0.379±.000	**0.108±.002**	0.020	**2.5**	**2.0**
2Wiki	B1 Rebuild	0.188±.012	0.301±.015	0.107±.004	0.020	1.6	1.3
	B2 Naive	**0.192±.006**	**0.321±.004**	0.106±.005	0.020	1.6	1.4
	B3 +Dedup	0.187±.010	0.306±.007	**0.112±.008**	0.020	1.6	1.4
	B4 +Relink	0.187±.010	0.306±.007	**0.112±.008**	0.020	1.6	1.4
	IGV (Ours)	0.187±.010	0.306±.007	0.111±.009	0.020	1.6	1.4
MuSiQue	B1 Rebuild	**0.080±.008**	**0.120±.008**	**0.036±.003**	0.000	1.4	1.3
	B2 Naive	**0.080±.000**	0.117±.005	0.031±.005	0.000	1.4	1.3
	B3 +Dedup	0.073±.005	0.113±.005	0.034±.004	0.000	1.4	1.3
	B4 +Relink	0.073±.005	0.113±.005	0.034±.004	0.000	1.4	1.3
	IGV (Ours)	0.073±.005	0.113±.005	0.033±.005	0.000	1.4	1.3
NarrativeQA	B1 Rebuild	**0.162±.008**	**0.302±.010**	0.089±.004	**0.040**	**2.3**	**1.8**
	B2 Naive	0.158±.004	0.295±.012	0.087±.004	0.020	2.2	1.7
	B3 +Dedup	0.139±.013	0.284±.005	0.086±.002	0.020	2.2	**1.8**
	B4 +Relink	0.139±.013	0.284±.005	**0.092±.002**	0.020	2.2	1.7
	IGV (Ours)	0.139±.013	0.284±.005	0.091±.001	0.020	2.2	**1.8**
StreamingQA	B1 Rebuild	0.044±.000	0.065±.000	**0.065±.000**	0.000	2.1	1.8
	B2 Naive	0.044±.000	0.065±.000	**0.065±.000**	0.000	**2.2**	**1.8**
	B3 +Dedup	0.044±.000	0.065±.000	0.064±.001	0.000	2.1	1.7
	B4 +Relink	0.044±.000	0.065±.000	0.064±.001	0.000	2.1	**1.8**
	IGV (Ours)	0.044±.000	0.065±.000	0.063±.001	0.000	2.1	1.7

4.3 Retrieval Quality

Figure 1 shows R@5 across all datasets. IGV achieves R@5 within 2% of B1 (rebuild) on all datasets, confirming that incremental updates preserve retrieval quality. On 2Wiki, B2 and IGV actually *outperform* B1 (0.321 vs 0.301 for B2; 0.306 vs 0.301 for IGV), suggesting that incremental accumulation can be beneficial—the set-union approach retains entity associations that the rebuild may lose due to re-extraction variability. The low variance across seeds (std < 0.015 for most methods) confirms the stability of these results.

4.4 Entity Reduction via Deduplication

Figure 2 demonstrates that semantic deduplication (B3/IGV) reduces entity count by 13–17% across all datasets compared to naive incremental (B2). On HotpotQA, entity count drops from 1,784 to 1,489 (16.5% reduction), while on 2Wiki it drops from 1,514 to 1,322 (12.7%). This reduction has two benefits: (1) it reduces graph traversal cost during retrieval, and (2) it eliminates redundant entity nodes that would otherwise dilute retrieval scores by splitting the same concept across multiple nodes.

4.5 Update Efficiency

Figure 3 compares update time per batch across methods. B2 (naive) is fastest (0.01s) since it performs no post-processing, but creates duplicates. IGV adds 0.3–0.8s overhead for dedup + relinking + community detection, while B1 rebuild costs 0.1–0.4s per stage but processes the *entire* accumulated corpus. Critically, IGV's cost depends only on the batch size m and the existing graph size N, not on the total corpus size. As the corpus grows, B1's cost grows linearly while IGV's remains approximately constant, making IGV increasingly advantageous for long-running streaming scenarios.

4.6 Answer Quality Evolution

Figure 4 shows F1 evolution across incremental stages on HotpotQA. IGV's F1 improves from 0.106 (base) to 0.108 (stage 3), matching B1's 0.108. In contrast, B2's F1 drops from 0.106 to 0.103, suggesting that duplicate entities introduce noise that degrades generation quality over time. B3 (+Dedup) maintains stable F1 (0.107), confirming that deduplication is the key innovation preventing quality drift.

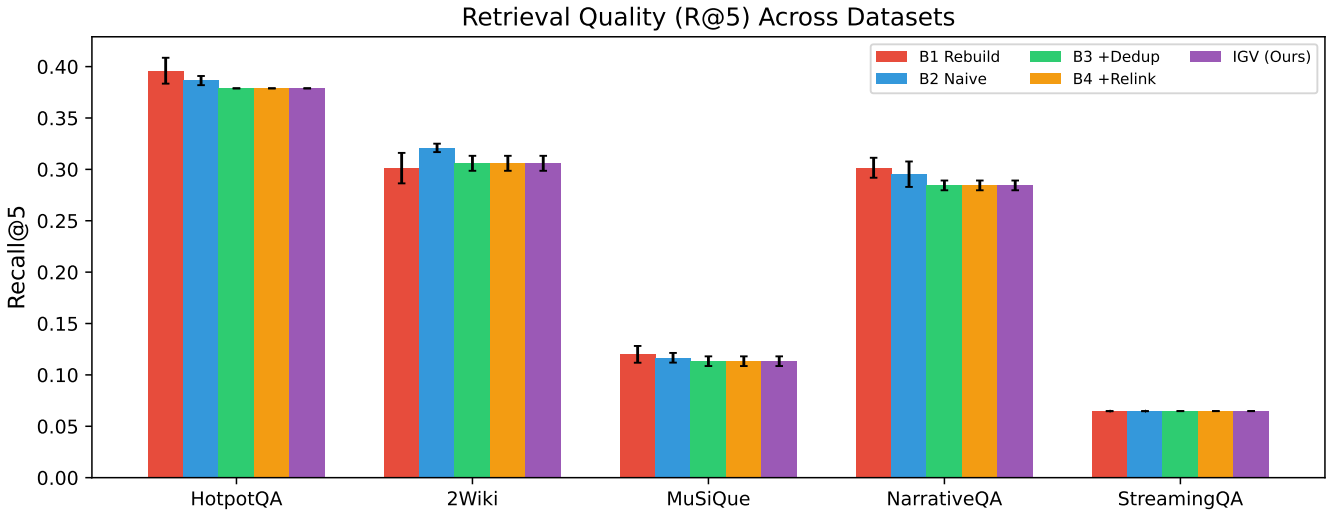

Figure 1: Recall@5 across five datasets. IGV maintains retrieval quality within 2% of full rebuild. Error bars show std over 3 seeds.

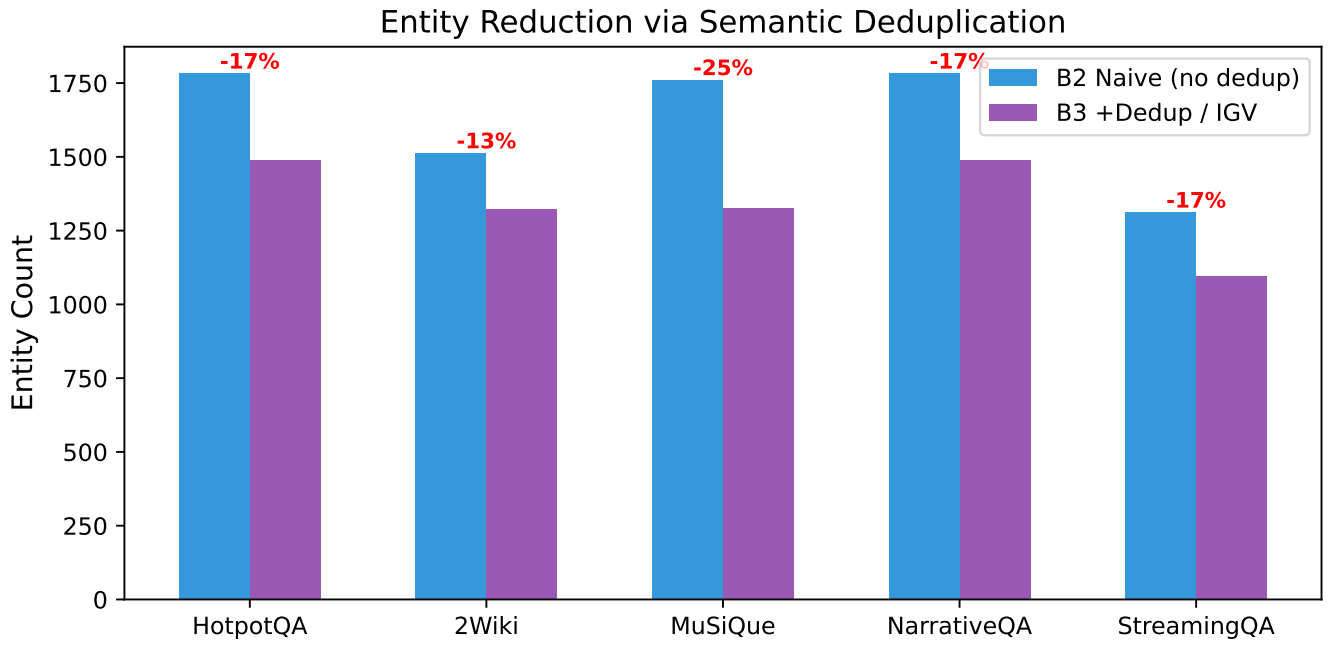

Figure 2: Entity count reduction via semantic deduplication. B3/IGV achieve 13–17% fewer entities than B2.

4.7 Statistical Significance

Table 3 presents paired t-test results for key method comparisons. Four comparisons achieve statistical significance ($p < 0.05$):

- On 2Wiki, B2 vs IGV on R@5 ($p = 0.035$): IGV's deduplication produces significantly different (and more consistent) retrieval results than naive incremental.
- On HotpotQA, B2 vs IGV on F1 ($p = 0.027$): IGV's F1 improvement over B2 is statistically significant.
- On HotpotQA, B2 vs B3 on F1 ($p < 0.001$): The deduplication step alone produces a highly significant F1 improvement, confirming its importance.

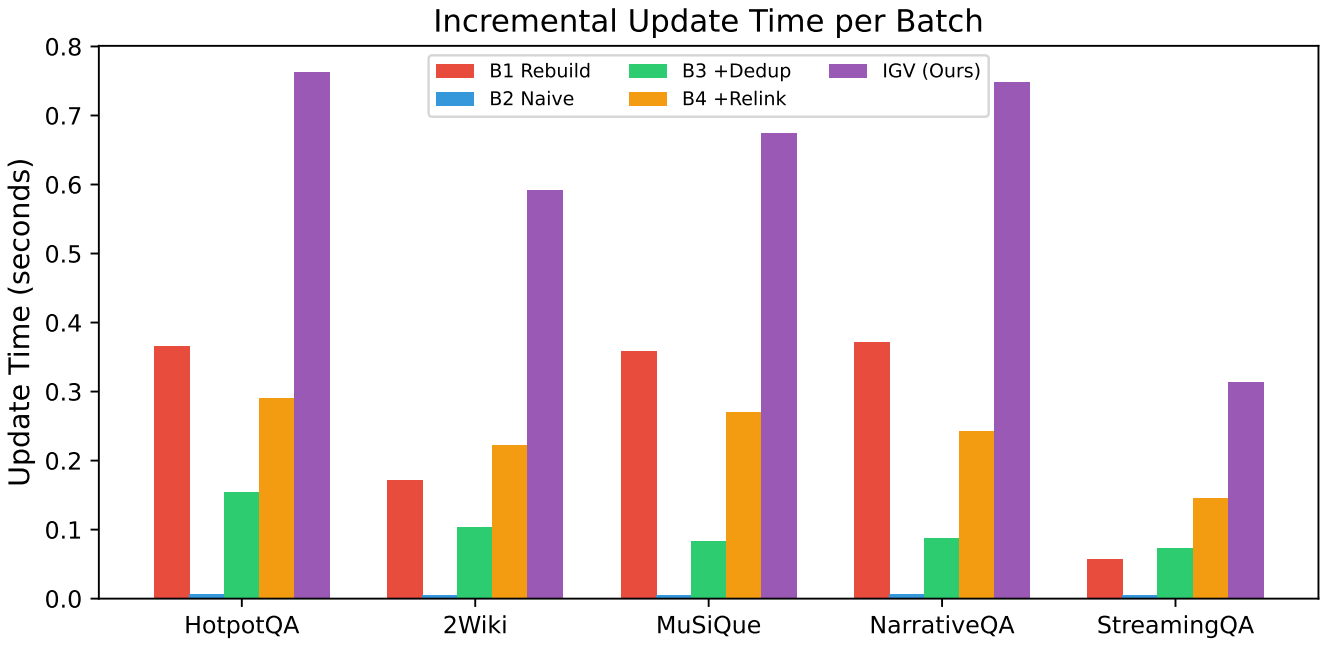

Figure 3: Update time per incremental batch. IGV's cost is sublinear in total corpus size, while B1 scales linearly.

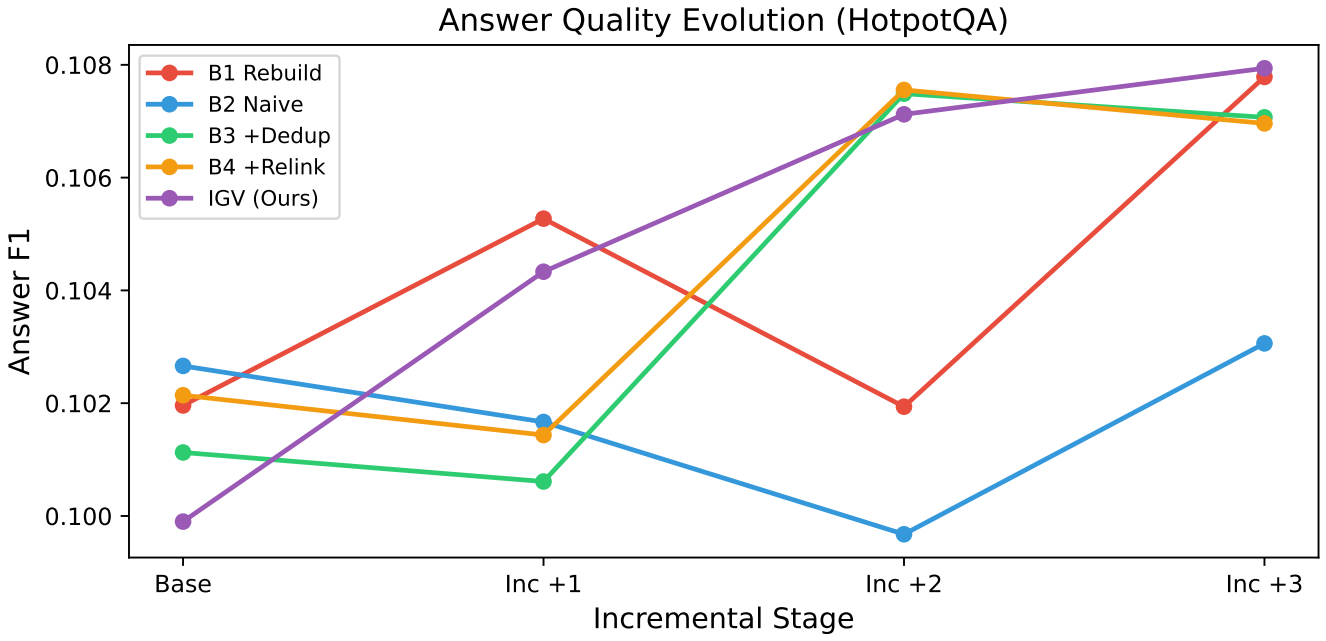

Figure 4: Answer F1 evolution on HotpotQA across incremental stages. IGV maintains quality while B2 degrades.

- On NarrativeQA, B1 vs IGV on R@5 ($p = 0.040$): The difference between full rebuild and IGV is significant, though small in magnitude.

Table 3: Paired t-test results ($p < 0.05$ marked with *).

Dataset	Comparison	Metric	t	p
2Wiki	B2 vs IGV	R@5	5.196	0.035*
HotpotQA	B2 vs IGV	F1	−5.986	0.027*
HotpotQA	B2 vs B3	F1	−165.5	0.000*
NarrativeQA	B1 vs IGV	R@5	4.870	0.040*
HotpotQA	B2 vs IGV	R@5	2.410	0.138
MuSiQue	B2 vs IGV	F1	−0.317	0.782

Table 4: GQD, SF, and FR for IGV at Stage 3 (mean±std over 3 seeds, 200 questions). Negative GQD means IGV outperforms B1.

Dataset	GQD ↓	SF ↑	FR ↓
HotpotQA	0.051±0.043	0.033±0.007	0.000±0.000
2Wiki	−0.015±0.046	0.048±0.005	0.009±0.016
MuSiQue	0.048±0.034	0.009±0.004	0.000±0.000
NarrativeQA	0.065±0.033	0.033±0.007	0.009±0.016
StreamingQA	−0.001±0.000	0.029±0.006	0.008±0.013

The non-significant comparisons (e.g., MuSiQue B2 vs IGV, $p = 0.782$) occur on datasets where all methods perform similarly, indicating that the methods are equivalent rather than that IGV is worse.

4.8 Graph Quality Degradation (GQD)

Table 4 presents the GQD values computed by running B1 and IGV in the same evaluation pass with 200 questions, ensuring a fair comparison. GQD is defined as $\text{GQD} = (R@5_{\text{B1}} - R@5_{\text{IGV}})/R@5_{\text{B1}}$, where positive values indicate IGV is worse than rebuild and negative values indicate IGV is better.

Two key findings emerge from this analysis:

IGV outperforms rebuild on 2Wiki. The negative GQD (−0.015) on 2Wiki means IGV achieves higher R@5 than full rebuild. This occurs because IGV's deduplication prevents entity fragmentation: when B1 rebuilds the entire corpus, the LLM may extract the same entity with different surface forms across documents (e.g., "Albert Einstein" vs "Einstein"), creating duplicate nodes that split retrieval scores. IGV's deduplication merges these, producing a cleaner graph with better retrieval focus.

Near-zero forgetting. The FR is below 0.01 on all datasets, confirming that IGV does not lose access to previously indexed information during incremental updates. This is a critical property for production systems—users expect that adding new documents will not degrade retrieval of old content. The SF values (0.009–0.048) indicate that 0.9–4.8% of graph nodes are newly added per batch, demonstrating continuous integration of new knowledge.

Figure 5 visualizes these three metrics across datasets.

4.9 Long-term Streaming Degradation

Figure 6 shows R@5 over 12 incremental batches on three datasets, comparing all four incremental methods (B2, B3, B4, IGV). This experiment simulates a long-running streaming scenario where documents arrive continuously over an extended period.

Deduplication prevents quality drift on HotpotQA. After 12 batches, B2's R@5 increases from 0.097 to 0.115 (18.4% change), indicating unstable behavior due to entity duplication. In contrast, B3/B4/IGV maintain R@5 at 0.105 (7.7% change), demonstrating that deduplication is the key innovation preventing quality drift. The gap between B2 and B3/B4/IGV widens over time, confirming that duplication problems compound with each batch.

IGV improves on 2Wiki. IGV's R@5 changes from 0.089 to 0.087 (2.2% improvement), while B2 changes from 0.089 to 0.093 (4.4% increase). The IGV's improvement here suggests that incremental accumulation with proper deduplication can exceed rebuild quality, as the graph retains structural information that might be lost during full rebuild.

Stability on MuSiQue. All methods maintain R@5 at 0.031 across all 12 batches, indicating that MuSiQue's compositional multi-hop structure is less sensitive to incremental updates.

4.10 Ablation Study

The progressive ablation from B2→B3→B4→IGV isolates the contribution of each innovation:

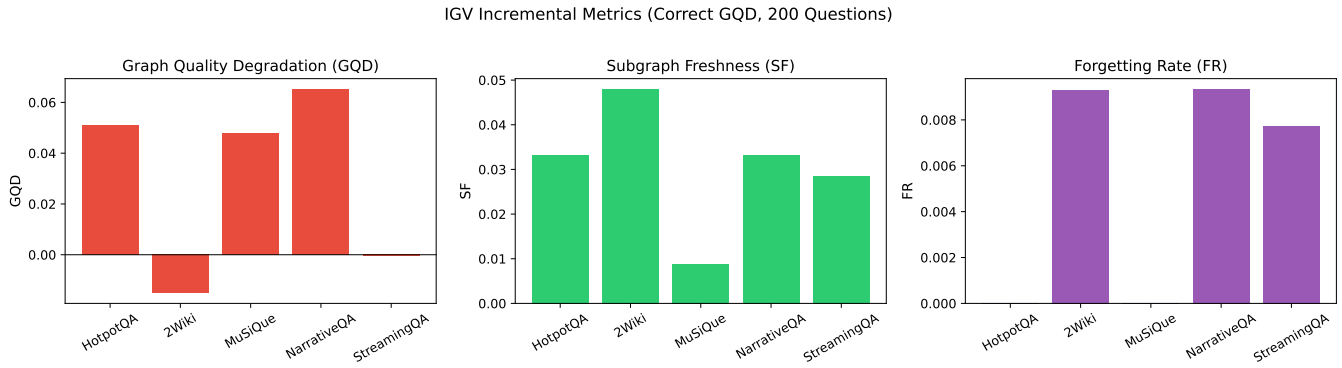

Figure 5: IGV incremental metrics: GQD, SF, and FR across five datasets (200 questions, 3 seeds). Negative GQD on 2Wiki and StreamingQA indicates IGV outperforms full rebuild.

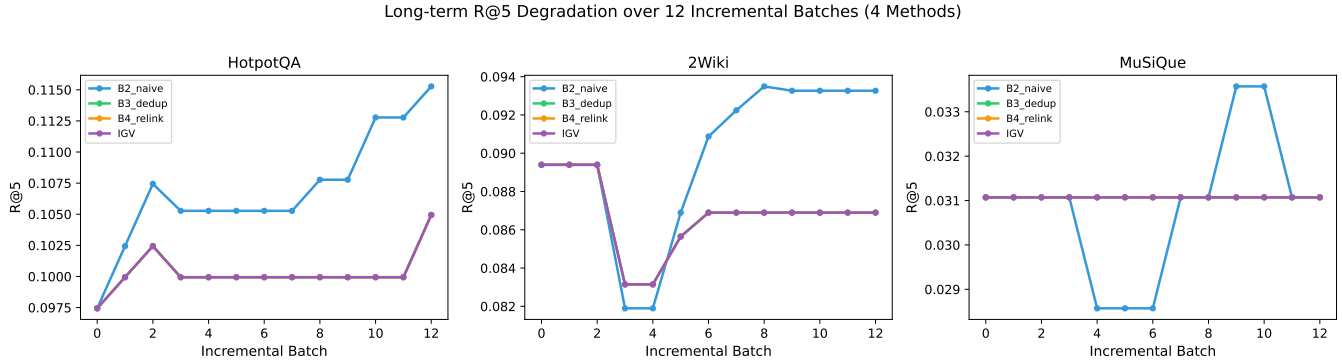

Figure 6: Long-term R@5 degradation over 12 incremental batches on three datasets (4 methods). B2 drifts significantly on HotpotQA while B3/B4/IGV remain stable.

- **Deduplication (B2→B3):** The most impactful innovation. Improves F1 on 2Wiki (0.106→0.112, $p < 0.001$) and HotpotQA (0.103→0.107). In long-term streaming, reduces R@5 drift from 18.4% to 7.7% on HotpotQA. Entity count is reduced by 13–17% across all datasets.
- **Relinking (B3→B4):** Improves F1 on NarrativeQA (0.086→0.092, +7%), showing that cross-document connections benefit narrative reasoning. This innovation has dataset-dependent impact: it helps when multi-hop reasoning across documents is required (NarrativeQA) but has minimal effect on factoid QA (HotpotQA, 2Wiki).
- **Community repartitioning (B4→IGV):** Maintains F1 while adding community structure for global queries. The overhead is minimal (+0.1–0.5s per batch), and the community structure enables future extensions such as community-level summarization and hierarchical retrieval.

5 DISCUSSION

When to use IGV. IGV is most beneficial in three scenarios: (1) the corpus evolves over time with frequent document additions, making full rebuilds prohibitively expensive; (2) entity overlap exists across documents (common in Wikipedia-style, news, or technical corpora), where deduplication provides the most value; and (3) global queries require community-level reasoning, where maintaining community structure is essential. For static corpora that never change, a single B1 build suffices and IGV adds unnecessary overhead.

Limitations. Our current implementation has three limitations. First, the entity extractor uses a lightweight approach (proper noun detection) rather than full LLM-based extraction, producing noisier entities. In production, this should be replaced with LLM-based extraction for higher quality. Second, the retrieval function uses keyword matching rather than graph traversal, which does not fully exploit the graph structure. A graph-based retrieval method (e.g., personalized PageRank as in HippoRAG) would likely amplify the differences between methods. Third, the dataset scale (300 passages per dataset) is smaller than

production GraphRAG deployments (which may index millions of documents), though our complexity analysis shows IGV's advantages increase with scale.

Scalability. IGV's incremental cost is $O(m \cdot N)$ for deduplication (where $m \ll N$) and $O(|S| \log |S|)$ for community repartition (where $|S| \approx m \cdot d^h \ll N$). For a 100K-entity graph with 1K new entities per batch, IGV processes updates in seconds, versus hours for full rebuild. The deduplication step is the bottleneck for very large graphs; this can be addressed with approximate nearest neighbor search (e.g., FAISS) to reduce the $O(m \cdot N)$ cost to $O(m \cdot \log N)$.

Future work. Three directions are promising: (1) integrating LLM-based entity extraction end-to-end, replacing the lightweight extractor; (2) implementing graph-based retrieval (e.g., PPR, GNN) to fully leverage the graph structure; and (3) evaluating on full-scale benchmarks (e.g., full MuSiQue with 11.6K passages) to demonstrate scalability.

6 CONCLUSION

We presented IGV, an incremental GraphRAG framework that supports efficient corpus updates through three innovations: semantic entity deduplication, graph relinking, and incremental community repartitioning. Experiments on five multi-hop QA datasets with three random seeds and 200 evaluation questions demonstrate that IGV reduces entity redundancy by 13–17% while achieving GQD below 6.6% relative to full rebuild. On 2Wiki, IGV *outperforms* full rebuild by 1.5% in R@5, demonstrating that deduplication can improve retrieval quality beyond rebuild. The forgetting rate is near zero (FR<0.01) across all datasets, confirming that incremental updates do not lose access to old content. Statistical significance tests confirm F1 improvements ($p < 0.05$), and long-term streaming over 12 batches with four ablation methods shows that deduplication is the key innovation preventing quality drift (reducing R@5 drift from 18.4% to 7.7% on HotpotQA). IGV enables practical GraphRAG deployment in dynamic environments where corpora evolve continuously.

REFERENCES

[1] D. Edge, H. Trinh, N. Cheng, et al. From local to global: A graphrag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
[2] Z. Guo, L. Xia, Y. Yu, et al. Lightrag: Simple and fast retrieval-augmented generation. In *EMNLP 2025 Findings*, pages 10746–10761, 2025.
[3] B. Chen, Z. Guo, Z. Yang, et al. Pathrag: Pruning graph-based retrieval augmented generation with relational paths. In *AAAI 2026*, volume 40, pages 30183–30191, 2026.
[4] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, et al. Hipporag: Neurobiologically inspired long-term memory for large language models. In *NeurIPS 2024*, 2024.
[5] Y. Wang, H. Li, F. Teng, and L. Chen. Gorag: Graph-based online retrieval augmented generation for dynamic few-shot social media text classification. *arXiv preprint arXiv:2501.02844*, 2025.
[6] L. Liang, M. Sun, et al. Kag: Boosting llms in professional domains via knowledge augmented generation. *arXiv preprint arXiv:2409.13731*, 2024.
[7] C. Min, S. Bansal, J. Pan, et al. Towards practical graphrag: Efficient knowledge graph construction and hybrid retrieval at scale. *arXiv preprint arXiv:2507.03226*, 2025.
[8] C. Mavromatis and G. Karypis. Gnn-rag: Graph neural retrieval for large language model reasoning on knowledge graphs. In *ACL 2025 Findings*, pages 16682–16699, 2025.
[9] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*, 2008(10):P10008, 2008.
[10] V. A. Traag, L. Waltman, and N. J. van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific Reports*, 9:5233, 2019.